운영체제 3주차: 프로세스와 스레드

강의 목표
- 프로세스와 스레드의 차이를 설명한다.
- CPU 스케줄링 알고리즘의 장단점을 비교한다.
- 교착 상태의 조건과 해결 방법을 이해한다.

프로세스의 개념

프로세스는 실행 중인 프로그램이다. 프로세스는 코드 영역, 데이터 영역, 힙, 스택으로 이루어진 독립된 주소 공간을 가진다. 운영체제는 각 프로세스를 프로세스 제어 블록(PCB)으로 관리하며, PCB에는 프로세스 ID, 프로세스 상태, 프로그램 카운터, 레지스터 값, 스케줄링 정보, 열린 파일 목록이 저장된다.

프로세스 상태는 생성(new), 준비(ready), 실행(running), 대기(waiting), 종료(terminated)의 다섯 가지로 나눈다.

스레드와 멀티스레딩

스레드는 프로세스 안에서 실행되는 흐름의 단위다. 같은 프로세스의 스레드들은 코드, 데이터, 힙과 열린 파일을 공유하지만 스택과 레지스터는 각자 가진다. 그래서 스레드 생성과 문맥 교환은 프로세스보다 비용이 작다.

멀티스레딩의 장점은 응답성 향상, 자원 공유, 경제성, 멀티코어 활용이다. 단점은 공유 데이터 접근 시 동기화가 필요하다는 점이며, 한 스레드의 오류가 프로세스 전체를 종료시킬 수 있다.

문맥 교환 (Context Switch)

문맥 교환은 CPU가 실행 중인 프로세스를 바꿀 때 현재 프로세스의 상태를 PCB에 저장하고 다음 프로세스의 상태를 PCB에서 복원하는 과정이다. 문맥 교환 동안 CPU는 유용한 작업을 하지 못하므로 순수한 오버헤드다. 문맥 교환이 너무 잦으면 시스템 처리량이 떨어진다.

CPU 스케줄링: 선점형과 비선점형

선점형 스케줄링은 실행 중인 프로세스를 운영체제가 강제로 중단시키고 다른 프로세스에 CPU를 할당할 수 있는 방식이다. 응답 시간이 짧아지지만 문맥 교환이 늘어난다.

비선점형 스케줄링은 프로세스가 종료되거나 입출력 요청으로 스스로 CPU를 반납할 때까지 기다리는 방식이다. 구현이 단순하지만 긴 작업이 CPU를 오래 점유하면 짧은 작업이 오래 기다린다.

스케줄링 알고리즘 비교

FCFS(선입선출)는 도착 순서대로 처리하며 호위 효과(convoy effect)가 생길 수 있다.

SJF(최단 작업 우선)는 평균 대기 시간을 최소화하지만 실행 시간을 미리 알기 어렵다.

라운드 로빈(Round Robin)은 시간 할당량(time quantum)만큼씩 돌아가며 실행한다. 시간 할당량이 너무 크면 FCFS와 같아지고, 너무 작으면 문맥 교환 오버헤드가 커진다. 일반적으로 10~100ms를 사용한다.

우선순위 스케줄링은 우선순위가 낮은 프로세스가 무한히 기다리는 기아(starvation)가 생길 수 있으며, 기다린 시간에 따라 우선순위를 높이는 에이징(aging)으로 해결한다.

교착 상태 (Deadlock)

교착 상태는 둘 이상의 프로세스가 서로가 가진 자원을 기다리며 영원히 진행하지 못하는 상태다. 교착 상태가 발생하려면 다음 네 가지 조건이 동시에 성립해야 한다.
1. 상호 배제: 자원은 한 번에 한 프로세스만 사용한다.
2. 점유 대기: 자원을 가진 채 다른 자원을 기다린다.
3. 비선점: 다른 프로세스의 자원을 강제로 빼앗을 수 없다.
4. 순환 대기: 프로세스들이 원형으로 서로의 자원을 기다린다.

교착 상태 처리 방법

예방(prevention)은 네 조건 중 하나를 원천적으로 막는다. 회피(avoidance)는 자원 할당 전에 안전 상태인지 검사하며, 대표적으로 은행원 알고리즘(Banker's algorithm)이 있다. 탐지와 회복(detection and recovery)은 교착 상태를 허용하되 주기적으로 탐지해 프로세스를 종료하거나 자원을 선점한다. 많은 범용 운영체제는 교착 상태를 무시하는 타조 알고리즘을 택한다.

세마포어와 뮤텍스

임계 구역(critical section)은 공유 자원에 접근하는 코드 영역이다. 뮤텍스는 잠금을 획득한 스레드만 임계 구역에 들어가게 하는 이진 잠금이다. 세마포어는 정수 값을 가진 동기화 도구로 wait(P)와 signal(V) 연산을 제공한다. 카운팅 세마포어는 여러 개의 동일 자원을 관리할 때 쓴다.

예시 코드:

```
  wait(S);
  /* critical section */
  signal(S);
```

가상 메모리와 페이지 교체

가상 메모리는 프로세스 전체를 메모리에 올리지 않고도 실행할 수 있게 한다. 필요한 페이지가 메모리에 없으면 페이지 폴트(page fault)가 발생한다.

페이지 교체 알고리즘: FIFO는 가장 먼저 들어온 페이지를 교체하며 벨레이디의 모순이 생길 수 있다. LRU는 가장 오랫동안 사용하지 않은 페이지를 교체한다. OPT(최적)는 앞으로 가장 오래 사용하지 않을 페이지를 교체하지만 미래를 알아야 하므로 비교 기준으로만 쓴다.

스래싱(thrashing)은 페이지 폴트가 너무 잦아 CPU 이용률이 급격히 떨어지는 현상이다.